# Curso
# AI no ciclo de software

*Prof. Luiz Calaça @luizcalaca*

# Quem sou: Luiz Calaça

- Desenvolvedor/Arquiteto 15+
- Professor (cursos livres, graduação e pós-graduação)
- Escritor
- Palestrante
- Consultor Brasil e Vale do Silício (AirApps e Tess)
- Já atuou com ex-Twitters

# Agenda

- 8:00 - 9:30h
- Intervalo - 9:30h - 9:45h
- 9:45h - 11:15h
- Intervalo 11:15h - 11:30h
- Exercício Case 11:15h - 12h

Ciclo de Software, Fundamentos e evolução da IA, Contexto Window, Long Running, Processamento LLM, Memória persistente no Claude.md, Engenharia de Contexto, Alucinação, Harness, Modelos, Skills, Máquina de estado, Review de código, Jailbreak, Subagents

# Ciclo de software

- XP, Espiral..
- Requisitos -> Implementação -> Testes -> Deploy
- Requisitos -> Testes (TDD) -> Implementação -> Deploy
- Deploy Blue Green

# Ciclo de software: onde está a memória do time?

“Eu já falei para usar o conventional commits”

“Alguém sabe como é o corpo da requisição de tal endpoint?”

“Essa parte do código eu nunca mexi, tem de ser com o Dev tal”

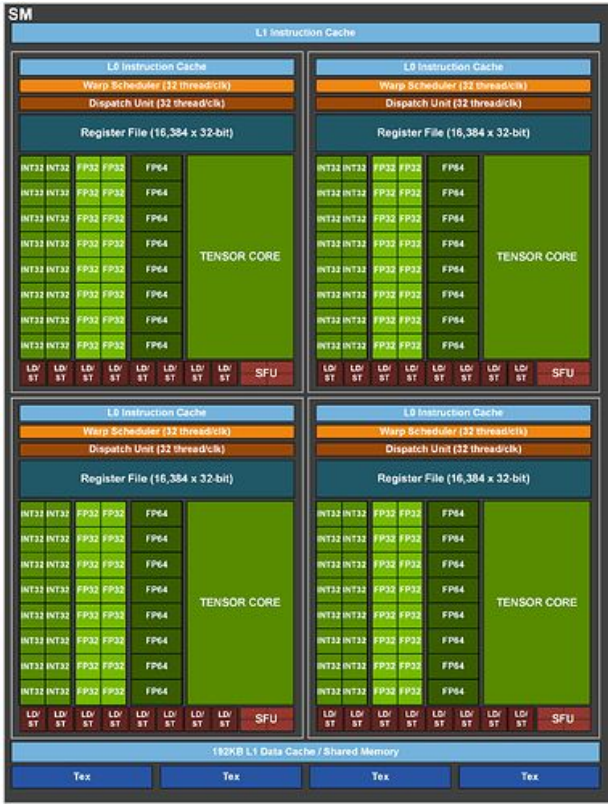
“Qual arquitetura sigo para tal tarefa..”

“Devo desenvolver com TDD, fazer testes unitários, BDD..”

“Como fazer o onboarding no projeto..”

# A IA

- Placas de vídeo (CUDA e computação paralela/distribuída)
- Redes neurais (Deep learning)
- NLP (Natural Language Processing)
- Imagem se tornam matrizes
- Áudio se torna uma imagem, que, pode ser transformar em matrizes
- PDF precisa ser lido, transformado em texto, feito embeddings e depois ser processado.



# Prompt Engineering: continua sendo importante!

- One shot
- Chain of thoughts
- Just repeat the same prompt in one chat: https://arxiv.org/abs/2512.14982

Inclusive, ele será importante para compor bem uma Skill

# O token

O que é?

Previsão do próximo token

Como observar o seu uso

# O token

**Tokenização:** seu texto é dividido em pequenas unidades chamadas *tokens*. Uma palavra pode corresponder a um token ou a vários.

**Representação numérica:** cada token é transformado em vetores de números (*embeddings*), que permitem ao modelo representar relações entre palavras e conceitos.

**Transformer e atenção:** esses vetores passam por várias camadas de uma arquitetura chamada **Transformer**. Um mecanismo central é a **attention**, que permite ao modelo determinar quais partes do contexto são mais relevantes. Em “João colocou o livro na mochila porque **ela** estava vazia”, por exemplo, o modelo usa o contexto para relacionar “ela” a “mochila”.

**Previsão do próximo token:** depois dos cálculos, o modelo produz uma distribuição de probabilidades para o próximo token:

# Processamento de um texto na LLM

**Texto**

↓

**Tokenização**

↓

**Embeddings**

↓

**Transformer**

→ Self-Attention

→ Feed Forward Network

→ Residual + Normalização

↓

**Probabilidades**

↓

**Próximo Token**

↓

↺ **Repete o processo**

# A Engenharia de contexto

**Engenharia de Contexto (Context Engineering)** é a prática de **projetar, organizar, selecionar e atualizar as informações que uma LLM recebe em seu contexto para aumentar a qualidade e a confiabilidade das respostas ou ações**.

Ainda, poderíamos dizer que o Context Engineering é a engenharia do contexto fornecido ao modelo para que ele tenha, no momento certo, as informações, instruções, ferramentas e estado necessários para executar uma tarefa.

***CUIDADO! Aumentar demais o contexto em um modelo pequeno pode piorar o resultado.***

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

# Context Window

- A janela de contexto inclui o input e o output
- O mesmo chat acumula contexto e o injeta automaticamente até fazer o /clear

https://platform.claude.com/docs/en/build-with-claude/context-windows

# A alucinação

Há uma limitação importante: como fundamentalmente está fazendo previsões, uma LLM pode produzir algo linguisticamente convincente **e ainda assim estar errada** — o fenômeno frequentemente chamado de *alucinação*.

**Prompt Engineering** escreve a instrução.

**Context Engineering** projeta o contexto.

**Agent Engineering** projeta o ciclo de decisão e ação.

# O tal Harness

É o esforço que se faz para que o agent compreenda a memória persistente do projeto e consiga gerar código confiável para a evolução contínua, boa manutenibilidade e para produção.

# O Harness

**Capacidades de execução:** Permitem que o agente interaja diretamente com o ambiente, realizando tarefas como manipular arquivos, executar instruções no sistema, modificar código e consumir serviços ou APIs externas.

**Gerenciamento de contexto e memória:** Responsável por organizar as informações relevantes durante a execução, preservando o estado da atividade e administrando o volume de dados para evitar que os limites de contexto sejam ultrapassados.

**Mecanismos de verificação e iteração:** Avaliam os resultados obtidos após cada etapa, identificando falhas ou inconsistências e possibilitando que o agente ajuste sua estratégia e tente novamente antes de avançar.

**Restrições, políticas e autorizações:** Definem quais operações podem ser realizadas pelo agente, estabelecendo regras de acesso, limites de atuação e condições de segurança para controlar sua interação com o ambiente.

# Evoluir um projeto é: documentá-lo!

Specification

Devo parar agora e documentar?

# Evoluir um projeto é: documentá-lo! E documentar o quê?

Tudo!

Regras de negócio

Arquitetura do software

APIs externas

Terraform/Cloud

Banco de dados

..

# Importante: diga também o que não deve ser feito

Você também deve salientar o que não deve ser feito (GuardRails)

“Dê um alerta ao tentar alterar um arquivo de configuração do banco de dados, ou sobre o pool, ou no Terraform (..)”

# O harness e o attention

Analogia: Queremos dar atenção ao que configuramos no Claude, a fim de que seja seguido.

# Os modelos: quais utilizarmos?

https://academy.claude.com/tutorials/choosing-the-right-claude-model

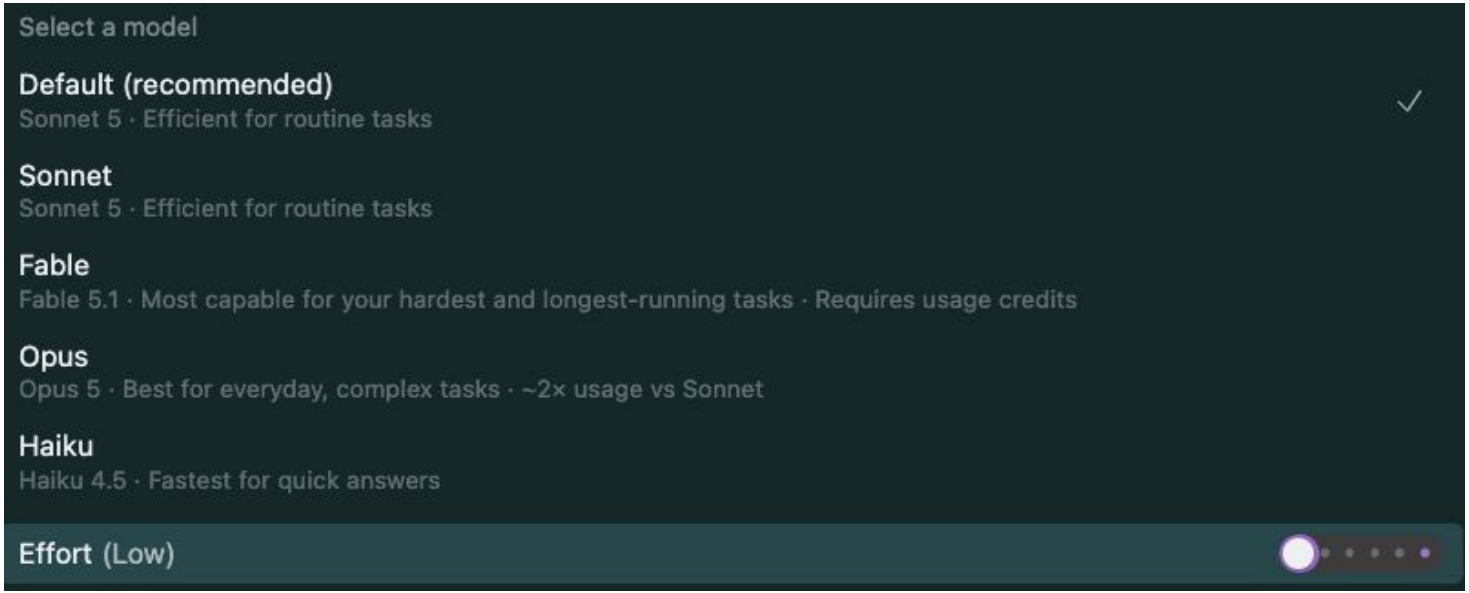
| Model | Rate limit use | Best for |
|---|---|---|
| Haiku | Lightest | Quick answers, summaries, and simple extraction — anything you want done instantly |
| Sonnet | Moderate | Coding, writing, analysis, and multi-step workflows — your versatile default |
| Opus | Heavy | Deep research and complex reasoning that genuinely needs sustained thinking |
| Fable | Heaviest | Your largest, most critical projects: long, complex tasks Claude works through with fewer check-ins |

# Os modelos: effort e reasoning

https://academy.claude.com/tutorials/choosing-the-right-claude-model

https://academy.claude.com/tutorials/choosing-the-right-claude-model#when-to-use-each-model

REASONING: analisar | decompor | comparar | testar hipóteses | verificar



# Três termos importantes

**Reasoning:** Capacidade/processo de raciocinar sobre um problema

**Thinking:** O processamento interno usado durante esse raciocínio

**Effort:** Quanto recurso/processamento o modelo é instruído a investir

# Planejamento de tarefas
# Os modelos: Um planeja e o outro faz a crítica?

Pedir o plan para um e a crítica a outro, vale a pena?

Isso pode ajudar a encontrar:

- requisitos esquecidos;
- edge cases;
- problemas arquiteturais;
- inconsistências;
- riscos de segurança;
- complexidade desnecessária;
- impactos no banco;
- problemas de escalabilidade;
- testes que estão faltando.

## Compare models

If you're unsure which model to use, start with Claude Opus 5.5 for most workloads. Use Claude Fable 5.1 for demanding reasoning and long-horizon agentic work, or when your evals on Claude Opus 5.5 at higher effort still fall short. All current models support text and image input, text output, multilingual capabilities, vision, and tool use. Each model's page lists the platforms it's available on.

| | Claude Fable 5.1 | Claude Opus 5.5 | Claude Sonnet 5 | Claude Haiku 4.5 |
|---|---|---|---|---|
| | For demanding reasoning and long-horizon agentic work | For long-running agentic coding and knowledge work | The best combination of speed and intelligence | The fastest model with near-frontier intelligence |
| Latência comparativa | Mais lento | Moderada | Rápido | Mais rápida |
| Preços | US$ 10 / MTok de entrada<br>US$ 50 / MTok de saída | US$ 4 / MTok de entrada<br>US$ 20 / MTok de saída | US$ 2 / MTok de entrada<br>US$ 10 / MTok de saída | US$ 1 / MTok de entrada<br>US$ 5 / MTok de saída |
| ID da API Claude | `claude-fable-5-1` | `claude-opus-5-5` | `claude-sonnet-5` | `claude-haiku-4-5-20251001` |
| **Capacidades** | | | | |
| Raciocínio | Adaptativo (sempre ativo) | Adaptativo (sempre ativo) | Adaptativo | Estendido |
| Esforço padrão | `high` | `medium` | `high` | Não compatível |
| Janela de contexto | 1 mi tokens | 1 mi tokens | 1 mi tokens | 200 mil tokens |
| Output máximo | 128 mil tokens | 128 mil tokens | 128 mil tokens | 64 mil tokens |
| Data de corte de conhecimento confiável | jun. de 2026 | jun. de 2026 | jan. de 2026 | fev. de 2025 |

> Detalhes adicionais

https://platform.claude.com/docs/en/models/overview#latest-models-comparison

# A utilização de subagents

Quando vale a pena?

Situações de refatoração em várias partes, pedindo para que os subagents possam analisar cada área: use cases, controllers e ORM, por exemplo.

Busca por bugs.

# Claude Code

https://support.claude.com/pt/articles/14553240-de-contexto-ao-claude-claude-md-e-prompts-melhores



# Dê contexto ao Claude: CLAUDE.md e prompts melhores

14 de abril de 2026

Copiar para LLM

Claude Code funciona bem pronto para usar, mas se torna notavelmente mais eficaz quando conhece as convenções do seu projeto e quando você adota alguns hábitos de prompt. Este guia cobre ambos.

## Parte 1 — CLAUDE.md: a memória do seu projeto

### O que é

`CLAUDE.md` é um arquivo markdown simples que Claude lê automaticamente no início de **cada** sessão naquele diretório. Pense nele como o briefing que você daria a um novo colega capaz no primeiro dia: como o time faz as coisas, o que evitar e onde as peças importantes vivem.

Você não precisa referenciá-lo em prompts ou anexá-lo manualmente. Se o arquivo existe, Claude já o leu.

### Onde fica

Claude procura em alguns lugares e mescla o que encontra, do amplo para o específico:

| Localização | Escopo | Bom para |
|---|---|---|
| `~/.claude/CLAUDE.md` | Cada projeto na sua máquina | Preferências pessoais (por exemplo, "uso pnpm, não npm" ou "sempre sugira testes"). |
| `<repo-root>/CLAUDE.md` | Este projeto | Arquitetura, convenções e comandos. **Este é o principal.** |
| `<subdir>/CLAUDE.md` | Aquele subdiretório (carregado sob demanda quando Claude lê arquivos naquele diretório, não no início da sessão) | Regras específicas do módulo (por exemplo, convenções diferentes em `frontend/` vs `api/`). |

A maioria dos times precisa apenas do arquivo na raiz do projeto. Faça commit no git para que todo o time se beneficie.

# Software

*Separação de responsabilidades na arquitetura*

*Testes, no contexto de IA, são mais obrigatórios ainda.*

*DDD, Clean Arch, SOLID, DRY, KISS*

*Complexidade de queries e algoritmos*



## Dê contexto ao Claude: CLAUDE.md e prompts melhores

14 de abril de 2026

Copiar para LLM

Claude Code funciona bem pronto para usar, mas se torna notavelmente mais eficaz quando conhece as convenções do seu projeto e quando você adota alguns hábitos de prompt. Este guia cobre ambos.

### Parte 1 — CLAUDE.md: a memória do seu projeto

#### O que é

`CLAUDE.md` é um arquivo markdown simples que Claude lê automaticamente no início de **cada** sessão naquele diretório. Pense nele como o briefing que você daria a um novo colega capaz no primeiro dia: como o time faz as coisas, o que evitar e onde as peças importantes vivem.

Você não precisa referenciá-lo em prompts ou anexá-lo manualmente. Se o arquivo existe, Claude já o leu.

#### Onde fica

Claude procura em alguns lugares e mescla o que encontra, do amplo para o específico:

| Localização | Escopo | Bom para |
|---|---|---|
| `~/.claude/CLAUDE.md` | Cada projeto na sua máquina | Preferências pessoais (por exemplo, "uso pnpm, não npm" ou "sempre sugira testes"). |
| `<repo-root>/CLAUDE.md` | Este projeto | Arquitetura, convenções e comandos. **Este é o principal.** |
| `<subdir>/CLAUDE.md` | Aquele subdiretório (carregado sob demanda quando Claude lê arquivos naquele diretório, não no início da sessão) | Regras específicas do módulo (por exemplo, convenções diferentes em `frontend/` vs `api/`). |

A maioria dos times precisa apenas do arquivo na raiz do projeto. Faça commit no git para que todo o time se beneficie.

# Vamos para mais

# As Skills e a máquina de estados (determinístico)

- A importância de uma máquina de estados e o elemento determinístico dentro de uma skills
- Evitando alucinações
- Garantindo que todos os passos serão feitos.

```
import { createEngine } from './statecraft.mjs';
```

Uma skill deve ser um pequeno contexto contendo a melhor prática

https://arxiv.org/html/2607.07676v1

# As Skills e os MCPs

A integração é oportuna.

# O review de código

https://arxiv.org/html/2607.07676v1

Gerar muito código e ter review humano para double check?

Gerar pouco código e fazer reviews humanos?

Gerar muito código com review feito por skill/agent?

Gerar pouco código com review feito por skill/agent?

# O jailbreak, guardrails e seu monitoramento

- Em sua essência é: Ignorar as regras, quebrar as restrições.
- O uso de skills ajuda a não tê-lo

**Guardrails (barreiras de proteção)** são mecanismos que **limitam, verificam ou bloqueiam o comportamento de uma IA**, garantindo que ela opere dentro de regras definidas.

Aconteceu algo que era guardrails? Alerta! Imediato. Pois, todo o time está exposto a fazer o mesmo, precisa ser corrigido em formato prioritário.

# Em essência, todo o time trabalha para a melhoria do Harness

“Eu trabalho para melhorar a IA, a fim dela trabalhar melhor para mim”

# Vamos para mais

# O long running

Agents and subagents fazendo tarefas até um fim determinado

https://platform.claude.com/docs/en/build-with-claude/context-windows#context-awareness